

For loop

```
for(i = 0, i <= 10, i++){
    k = i*4;
}
```

For venus:

```
    add    x5,    x0,    x0    # init i
    add    x6,    x0,    x0    # init k
    addi   x7,    x0,    10    # init max
for_loop:
    bge     x5, x7,    exit
less_than:
    slli   x6,    x5,    2
    addi   x5,    x5,    1
    beq    x0,    x0,    for_loop
exit:
    addi   x5,    x0,    0    # reset i, k, max
    addi   x6,    x0,    0    # reset i, k, max
    addi   x7,    x0,    0    # reset i, k, max
```

Leaf:

```

divide_by_2(int i){           //x10 → i
    int k;                   // x5 → k (temp, does not need to be saved)
    k = i/2;                 // srli
    return k;                //x10 → return
}

```

```

divide_by_2:
    addi    sp,    sp,    -8    // 8 bytes
    sd      x5,    0(sp)      // push operation
    srli    x5,    x10,    1    // /2 operation, x10 contains i
    add     x10,   x0,    x5    // copy k(x5) as ret val, NOT saved on stack
    ld      x5,    0(sp)      // pop operation, restore x5
    addi    sp,    sp,    8    // restore stack pointer
    jalr    x0,    0(x1)      // x1 contains return address, while x10 contains ret val

```

For venus:

```

    addi    x10,   x0,    100    # init i
    addi    x5,    x0,    200    # init j
    addi    sp,    x0,    1024    # init sp
divide_by_2:
    addi    sp,    sp,    -4      # 8 bytes
    sw      x5,    0(sp)          # push operation
    srli    x5,    x10,    1      # /2 operation, x10 contains i
    add     x10,   x0,    x5      # copy k(x5) as ret val, NOT saved on stack
    lw      x5,    0(sp)          # pop operation, restore x5
    addi    sp,    sp,    4      # restore stack pointer
    jalr    x0,    0(x1)          # x1 contains return address, while x10 contains ret

```

Non-leaf

```
add_with_divide(int i, int j){      // x10 → i, x11 → j
    int y = 0. z = 0;              // x5 → y and x6 → z
    y = divide_by_2_fx(i);
    z = y + i + j;
    return z;                      // x10 → return
}

add_with_divide:
    addi    sp,    sp,    -24      // 8 bytes
    sd      x5,    0(sp)          // push operation
    sd      x6,    8(sp)          // push operation
    sd      x1,    16(sp)         // push operation
    add     x6,    x10, x11        // can do i+j
    jal     x1,    divide_by_2    // x1 changed here, also x10 will contain ret val'
    add     x5,    x10, x0         // optimization for compiler
    add     x6,    x6,  x10        // now i + j + y
    ld      x1,    16(sp)         // pop operation, restore x1
    ld      x6,    8(sp)          // pop operation, restore x6
    ld      x5,    0(sp)          // pop operation, restore x5
    addi    sp,    sp,    24      // restore stack pointer
    jalr    x0,    0(x1)          // x1 contains return address, while x10 contains ret val
```

For venus (nested procedure call):

```
addi x10, x0, 100      # init i
addi x5, x0, 200        # init j
addi sp, x0, 1024      # init sp
```

add_with_divide:

```
addi sp, sp, -12        # 4 bytes
sw x5, 0(sp)            # push operation
sw x6, 4(sp)            # push operation
sw x1, 8(sp)            # push operation
add x6, x10, x11        # can do i+j
jal x1, divide_by_2     # x1 changed here, also x10 will contain ret val'
add x5, x10, x0          # optimization for compiler
add x6, x6, x10          # now i + j + y
lw x1, 8(sp)            # pop operation, restore x1
lw x6, 4(sp)            # pop operation, restore x6
lw x5, 0(sp)            # pop operation, restore x5
addi sp, sp, 12         # restore stack pointer
jalr x0, 0(x1)          # x1 contains return address, while x10 contains ret val
```

divide_by_2:

```
addi sp, sp, -4         # 4 bytes, jumped
sw x5, 0(sp)            # push operation
srli x5, x10, 1         # /2 operation, x10 contains i
add x10, x0, x5         # copy k(x5) as ret val, NOT saved on stack
lw x5, 0(sp)            # pop operation, restore x5
addi sp, sp, 4          # restore stack pointer
jalr x0, 0(x1)          # x1 contains return address, while x10 contains ret
```

Swap procedure

```
swap(int *v, int k){           // x10 → v, x11 → k(offset)
    int temp;                  // x5 → temp
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;             // no return statment
}
```

swap:

```
slli    x6,    x11,  3        // x8-ing the offset
add     x6,    x10,  x6        // address = base + offset, &v[k]
addi    x7,    x6,    8        // next element, &v[k+1]
ld      x5,    0(x6)          // temp = v[k] = x5
ld      x8,    0(x7)          // x8 → v[k+1]
sd      x8,    0(x6)          // v[k] = v[k+1]
sd      x5,    0(x7)          // v[k+1] = temp
jalr    x0,    0(x1)          // jump back to PC+4
```